

NCCL M2N: A Layout- and Topology-Aware Collective for Distributed Tensor Resharding

Kaushik Kandadi, Youngeun Kwon, Sreeram Potluri, Ching-Hsiang Chu,
Ke Wen, Pouya Kousha, Sangkug Lym, Nitin Nitin, Manjunath Gorentla Venkata

NVIDIA Corporation

{kkandadisure, youngeunk, spotluri, chichu, kwen, pkousha, slym, nitin, manjunath}@nvidia.com

Abstract—Distributed deep learning training partitions tensors across a process mesh according to a parallelism configuration, producing a corresponding sharding and replication layout. Reinforcement learning with verifiable rewards (RLVR) often uses different layouts for policy training and rollout generation; crossing this boundary requires *resharding* from one layout to another: the *M-to-N layout redistribution* problem, in which M source ranks collectively hold a global tensor in one layout and N destination ranks need it in another. This pattern is absent from today’s collective libraries: flat direct sends duplicate every source contribution across destination replicas, while gather-then-broadcast concentrates traffic at one root and over-delivers the full tensor. These workarounds are costly: with legacy all-gather plus broadcast, RL weight sync accounts for 29.4% of the reported RL step time for DeepSeek-V3 at 256 GPUs.

We present NCCL M2N, a collective primitive implemented in NCCL Extensions for cross-layout sharded-tensor transfer. It automatically *derives the transfer schedule from the layout endpoints*. Compared with flat direct sends and gather-then-broadcast, the collective provides: (1) layout-derived traffic — only bytes required by the destination layout are transferred; (2) load-balanced NIC usage — source contributions are distributed across eligible destination leaders; and (3) topology-aware routing — replicated fan-out stays within an NVLink domain while a pipelined ring carries one copy between domains.

We evaluate NCCL M2N on up to 256 GB200 GPUs in an NVL72 cluster with NDR InfiniBand. On a single FFN-MoE layer transfer, NCCL M2N achieves up to $7.9\times$ speedup over flat direct sends (9.8 ms vs. 77.3 ms), closely tracking the aggregate data-movement bound. In a non-located DeepSeek-V3 NeMo-RL run, NCCL M2N reduces the reported weight-sync time from 5.78 s to 2.77 s ($2.09\times$), reducing step time by 12.7% over legacy all-gather plus broadcast.

Index Terms—collective communication, distributed deep learning, reinforcement learning, sharded tensor, resharding, NCCL, GPU interconnect

I. INTRODUCTION

Distributed deep learning training partitions model tensors across graphics processing units (GPUs) using various parallelism strategies such as data parallelism (DP), tensor parallelism (TP), pipeline parallelism (PP), and expert parallelism (EP) [1]–[3] to leverage the high-performance computing power of GPU clusters. As a result, these choices define each tensor’s *layout*: how the data are sharded or replicated across the GPUs in the so-called process mesh. Modern frameworks expose this abstraction as *distributed tensors*, including PyTorch DTENSOR [4], GSPMD (general and scalable parallelization for machine-learning computation

TABLE I
LEGACY ALL-GATHER-PLUS-BROADCAST WEIGHT-SYNC COST AS A FRACTION OF REPORTED RL STEP TIME.

Model	Scale	Refit share
DeepSeek-V3	256 GPUs	29.4%

graphs) [5], and Alpa [6]. Application code can then reason about a tensor’s global shape while the runtime manages local placement.

Modern large language model (LLM) workflows routinely require *resharding* tensors from one layout to another as parallelism changes across phases. In such an M-to-N layout reshard operation, M source ranks collectively hold a tensor in one layout and N destination ranks require it in another. The runtime must derive the overlapping tensor regions, deliver each region to every required replica, and coordinate ranks that do not belong to the same process group. RLVR-style training exposes this operation directly: it alternates rollout generation and policy updates, often using different parallel layouts for the generator and trainer. After each update, policy weights must be resharded and transferred across the process-group boundary before the next rollout begins. We refer to this synchronization phase as *weight refit*. Recent frameworks (NeMo-RL [7], OpenRLHF [8], and HybridFlow/verl [9]) expose such a phase. The RL weight refit step, which transfers updated policy weights from trainer to generator after each training step, is the most costly instance of this pattern. It becomes one of the major performance bottlenecks as it sits directly on the critical path, where generators cannot begin the next rollout batch until new weights arrive. Our profiling of DeepSeek-V3 at 256 GPUs showed that legacy all-gather plus broadcast weight sync accounted for 29.4% of the reported RL step time (Table I), a substantial share of the end-to-end step.

Despite this cost, today’s communication stack does not express this cross-group resharding operation natively. The resulting workarounds expose two primary limitations:

- (i) flat direct sends duplicate every source contribution across destination replicas; and
- (ii) gather-then-broadcast avoids source-side duplication but concentrates traffic at one root and over-delivers the full tensor, wasting network bandwidth.

To address these problems, this paper proposes a layout-derived reshard collective primitive and a topology-aware routing optimization. The collective derives one global transfer schedule from the source and destination layouts, including exact overlap regions, replica ownership, and disjoint process groups. The route sends one copy of each contribution per destination NVLink domain, balances injection across eligible NICs, forwards copies between domains, and performs replica fan-out locally over NVLink. These choices remove replica-multiplied network traffic while preserving the semantics of M-to-N layout changes expressible by the supported descriptors (§III, §V).

This paper makes the following contributions.

- (1) **A first-class M-to-N reshard collective primitive** with mesh-and-placement descriptors. It derives exact transfer regions across tensor dimensions, mismatched replica counts, cross-dimension sharding, and disjoint trainer/rollout process groups (Sections §III, §IV).
- (2) **A topology-aware routing optimization** that removes inter-node replica redundancy, balances NIC injection under shard imbalance, and performs local fan-out over NVLink while forwarding one copy across domains (Section §V).
- (3) **An evaluation of the hierarchical route** on an NVIDIA GB200 NVL72 cluster. The measured route achieved up to a $7.9\times$ speedup for a single FFN-MoE layer transfer (9.8 ms versus 77.3 ms). In a separate non-colocated 256-GPU DeepSeek-V3 NeMo-RL run, NCCL M2N reduced the reported weight-sync time by $2.09\times$ and reduced step time by 12.7% over legacy all-gather plus broadcast (Section §VI).

II. BACKGROUND

A. RL Training Loop and Refit

Reinforcement-learning fine-tuning alternates between rollout generation, policy training, and weight synchronization. The policy uses a parallel configuration optimized for training throughput and a separate generation configuration optimized for rollout throughput. At each synchronization point, updated weights move from the training layout to the generation layout. We call this in-memory transfer phase *refit*; it is an M-to-N layout redistribution [10].

Refit combines different tensor layouts with communication between potentially disjoint process groups and with destination-side replication. Because it runs between training and the next rollout, its transfer schedule affects the iteration critical path. The following subsections describe how frameworks express these layouts and why existing communication abstractions do not directly realize the transfer.

B. Distributed Tensor Layouts

Modern distributed-tensor frameworks can describe a global tensor with a *process mesh* and a per-tensor *placement*. A placement maps each mesh axis either to a tensor dimension (sharding) or to replication. PyTorch’s DTENSOR [4],

GSPMD [5], and Alpa [6] follow this model. For the materialized parameter tensors considered here, we use SHARD and REPLICATE placements. Given a global shape $\mathbf{S}$, a mesh M , and a placement list p whose entries are SHARD(d) or REPLICATE, each rank’s local shape and offset are determined deterministically. We call the tuple $(\mathbf{S}, M, p)$ a *layout*.

A *reshard* converts a tensor from a source layout $(\mathbf{S}, M_{\text{src}}, p_{\text{src}})$ to a destination layout $(\mathbf{S}, M_{\text{dst}}, p_{\text{dst}})$. When the source and destination groups contain M and N ranks, respectively, we call this an *M-to-N layout redistribution*. The two groups can use different shard dimensions, shard counts, and replication factors, and can be disjoint. The runtime must therefore derive which source regions overlap each destination region instead of relying on a fixed participant pattern.

Tensor Parallelism (TP) and Expert Parallelism (EP) shard tensor and expert dimensions, respectively. Pipeline Parallelism (PP) assigns successive layers to different stages, while Data Parallelism (DP) replicates a model or tensor. Combining these strategies gives training and generation layouts with different ownership and replication patterns. In particular, TP and EP may shard different dimensions, so a reshard can require intersecting multi-dimensional regions rather than copying one rank’s contiguous slice.

C. Collective Communication and Interconnect Hierarchy

GPU-side distributed training relies on the NVIDIA Collective Communications Library (NCCL) [11], which provides ALLREDUCE, ALLGATHER, REDUCESCATTER, BROADCAST, ALLTOALL, and point-to-point *send/recv*. A collective API fixes its participants and logical input-to-output pattern; the implementation can still choose topology-aware paths for that pattern. For example, ALLGATHER and ALLTOALL cover common in-mesh transformations for materialized layouts when membership and sharded dimensions align. REDUCESCATTER addresses a different transformation that combines partial values with reduction; that PARTIAL placement is outside the layout subset used here. These collectives do not accept two arbitrary layouts or derive the byte-level intersections needed when the source and destination groups differ. No standard collective therefore expresses the full cross-group M-to-N redistribution considered here.

NCCL also exposes lower-level mechanisms that can realize a custom route. LSA enables direct peer-memory loads and stores over supported peer links, including within an NVLink domain. GPU-Initiated Networking (GIN) [12] provides GPU-initiated puts and signals over one-sided Remote Direct Memory Access (RDMA), while host-initiated one-sided operations support staged transfers. These mechanisms move data and signals, but they do not determine which layout regions to send or where to create replicas; those decisions belong to the reshard schedule. Because intra-domain links are typically faster than cross-domain links, a topology-aware route should perform replica fan-out within the destination domain and send each required tensor region once into each destination domain.

III. MOTIVATION AND DESIGN CONSIDERATIONS

A. Limitations of Existing Approaches

Cross-layout transfers are often assembled from existing lower-level communication operations, with two primary limitations:

- (i) flat direct sends duplicate every source contribution across destination replicas; and
- (ii) gather-then-broadcast avoids source-side duplication but concentrates traffic at one root and over-delivers the full tensor, wasting network bandwidth.

The missing capability is therefore not another send operation. The runtime needs one collective that jointly plans the M-to-N redistribution from both endpoint layouts, identifies the overlapping tensor slices, maps each slice to source and destination byte offsets, and accounts for the communication topology.

B. Layout-Derived Schedule and Hierarchical Route

We define a *layout-derived schedule* as a transfer plan computed at call time from the source and destination layout descriptors together with the runtime topology. The descriptors determine which tensor regions overlap and which destination replicas require each region. The topology determines which leaders and links carry those regions.

For the layout class formalized in Section IV, the hierarchical route provides three properties. First, it removes replica-multiplied source egress by injecting each required contribution once and forwarding one copy through each destination domain. Second, it balances the number of source contributions assigned to each eligible leader rather than concentrating them on one receiver. Third, it keeps replica forwarding within an NVLink domain and uses inter-domain links only for the copies that must cross them. These properties eliminate replica-multiplied network traffic and single-ingress over-delivery while enabling global topology-aware load balancing and preserving the semantics of the supported M-to-N layouts (Section IV).

The next sections formalize the layout and overlap semantics, then derive the schedule and hierarchical route that realize these properties.

IV. RESHARD SEMANTICS AND DESCRIPTORS

To dive deeper into the semantics of resharding, we first introduce the layout model and relevant terminology as shown in Table II.

Process mesh: A *process mesh* M is a multidimensional grid of GPU ranks with a deterministic mapping from mesh coordinates to communicator ranks. For the two-dimensional $D_0 \times D_1$ case used in our examples, indices (i_0, i_1) select a slot with $0 \leq i_k < D_k$. The grid is anchored at a *base rank* r_0 within the communicator used for this reshared; slots form a contiguous row-major rank interval,

$$\text{rank}(i_0, i_1) = r_0 + i_0 D_1 + i_1. \quad (1)$$

TABLE II
NOTATION FOR THE LAYOUT MODEL.

Symbol	Meaning
$L = (\mathbf{S}, M, p)$	Complete layout comprising a global tensor shape, process mesh, and placement list.
$\mathbf{S} = (S_0, \dots, S_{n-1})$	Global tensor shape; S_d is the extent of tensor dimension d .
M, D_k, r_0	Process mesh; D_k is the number of ranks along mesh axis k , and r_0 is the mesh's base rank within the communicator.
i_k	Coordinate along mesh axis k , with $0 \leq i_k < D_k$.
$p_k = \text{SHARD}(d)$	Axis k partitions tensor dimension d into D_k contiguous pieces; each rank stores a $1/D_k$ slice along d .
$p_k = \text{REPLICATE}$	Axis k duplicates data: every rank along that axis holds the same slice; it does not subdivide any tensor dimension.

Placements and layouts: A *placement* list p fixes how each active mesh axis relates to tensor storage; a *layout* $L = (\mathbf{S}, M, p)$ pairs that mesh with a global shape $\mathbf{S} = (S_0, \dots, S_{n-1})$. Table II summarizes the layout notation.

Local shard bounds: To determine which portion of the global tensor a rank owns, consider the rank at mesh coordinates $\mathbf{i}$. Its *local shard* spans $[l_d, u_d)$ along each tensor dimension d . The lower endpoint is

$$l_d = \begin{cases} \lfloor i_k \cdot S_d / D_k \rfloor & \exists k : p_k = \text{SHARD}(d), \\ 0 & \text{otherwise (REPLICATE-only)}. \end{cases} \quad (2)$$

The upper endpoint u_d uses the same formula with (i_k+1) in place of i_k . Supported sharded dimensions divide evenly by their shard count; replicated dimensions use the full extent $[0, S_d)$. The public API accepts each side's local shape rather than $\mathbf{S}$ and verifies that the two descriptors imply the same global shape.

Reshard: A *reshard* from $L_s = (\mathbf{S}, M_s, p_s)$ to $L_d = (\mathbf{S}, M_d, p_d)$ is a collective that, when complete, leaves every destination rank $r \in M_d$ holding the local shard of $\mathbf{S}$ prescribed by L_d . The implementation derives the full (source rank, destination rank, byte-range) plan from the two endpoints; callers never supply a manual per-pair schedule. Every communicator rank supplies both endpoint layout descriptions, L_s and L_d , so that all ranks derive the same schedule. For each endpoint, a rank supplies a pointer to its local tensor buffer only if it participates in that endpoint's mesh; otherwise, only the layout metadata is required. The abstract layout model does not require disjoint meshes. The current implementation, however, requires M_s and M_d to be disjoint intervals within the supplied communicator.

Next, we use two running examples to illustrate the semantics of resharding. Example A highlights destination-side replication and same-dimension TP consolidation: a TP=4 trainer (DP=1) refitting weights to a TP=2, DP=2 generator. Example B shows how pipeline parallelism (PP > 1) is handled via per-stage decomposition: a TP=2, PP=2 trainer refitting weights to a TP=1, PP=1 generator.

Example A: same-dimension TP reshard with destination replicas: Let $\mathbf{S} = [4, 8]$. The source mesh places

TABLE III

SHARD OWNERSHIP FOR EXAMPLE A ($\mathbf{S} = [4, 8]$, MESH $[1 \times 4] \rightarrow [2 \times 2]$): SOURCE AND DESTINATION BOTH SHARD DIMENSION 0; EACH DESTINATION TP SLOT CONSOLIDATES TWO SOURCE TP SHARDS AND IS REPLICATED TWICE (DP=2). [A:B, C:D]: DIM-0/DIM-1 SLICE OF $\mathbf{S}$.

Rank	Role	Local shard	Parallelism
0	src	[0:1, 0:8]	TP=0
1	src	[1:2, 0:8]	TP=1
2	src	[2:3, 0:8]	TP=2
3	src	[3:4, 0:8]	TP=3
4	dst	[0:2, 0:8]	TP=0, DP=0
5	dst	[2:4, 0:8]	TP=1, DP=0
6	dst	[0:2, 0:8]	TP=0, DP=1 (replica of rank 4)
7	dst	[2:4, 0:8]	TP=1, DP=1 (replica of rank 5)

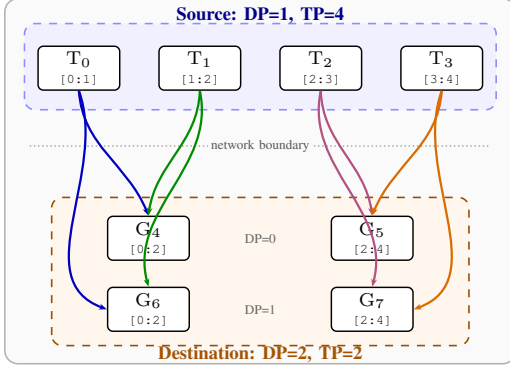


Fig. 1. Flat direct-send realization of Example A's required ownership. Each source contribution is sent to both destination DP replicas, so the route transfers $2B$ over the network. Colors distinguish the four source contributions.

four ranks $\{0-3\}$ in a $[1 \times 4]$ grid ($r_0^s = 0$) with $p_s = (\text{REPLICATE}, \text{SHARD}(0))$: axis-0 replicates trivially (DP=1) and axis-1 shards dimension 0 into four equal slices (TP=4), so each source rank holds one row of $\mathbf{S}$. The destination mesh places four ranks $\{4-7\}$ in a $[2 \times 2]$ grid ($r_0^d = 4$) with $p_d = (\text{REPLICATE}, \text{SHARD}(0))$: axis-0 replicates (DP=2) and axis-1 shards dimension 0 into two slices (TP=2). Table III and Figure 1 show the resulting shard ownership and mesh layout.

Since both meshes shard along dimension 0, each destination TP slot consolidates two adjacent source TP shards: G_4 receives rows $[0:1]$ and $[1:2]$ from T_0 and T_1 ; G_5 receives rows $[2:3]$ and $[3:4]$ from T_2 and T_3 . Destination row 1 (G_6, G_7) holds data identical to row 0 (generator-side DP=2 replication): G_6 duplicates G_4 and G_7 duplicates G_5 . Figure 1 shows a flat direct-send realization of this ownership: each source contribution is sent to both DP copies. This transfers two copies of the unique payload over the network and motivates the topology-aware route described in §V-B.

Example B: PP > 1 via per-stage reshard decomposition. With pipeline parallelism ($PP > 1$), model layers are partitioned across PP stages; each stage's ranks own weights for only their assigned layers. Because different stages hold entirely different weight tensors, no single reshard can span the

TABLE IV

SHARD OWNERSHIP FOR EXAMPLE B ($\mathbf{S} = [4, 8]$, TRAINER TP = 2 PP = 2, GENERATOR TP = 1 PP = 1). EACH RESHARD USES ITS OWN PP COMMUNICATOR; C-RANK IS THE RANK'S LOCAL INDEX WITHIN THAT COMMUNICATOR. RANK 4 (GLOBAL) JOINS BOTH PP COMMS AS C-RANK 2, ACCUMULATING ONE FULL-SHAPE TENSOR PER STAGE (S0/S1).

Rank	Role	PP comm	c-rank	Local shard	TP slot
<i>Reshard call 1 — PP comm 0 {ranks 0, 1, 4}</i>					
0	src	0	0	[0:2, 0:8]	TP=0
1	src	0	1	[2:4, 0:8]	TP=1
4	dst	0	2	[0:4, 0:8]	full (s0)
<i>Reshard call 2 — PP comm 1 {ranks 2, 3, 4}</i>					
2	src	1	0	[0:2, 0:8]	TP=0
3	src	1	1	[2:4, 0:8]	TP=1
4	dst	1	2	[0:4, 0:8]	full (s1)

full trainer mesh—instead, the caller issues *one reshard per PP stage*, each scoped to that stage's ranks plus the generator, leaving the reshard primitive itself unchanged.

Consider four trainer ranks (TP = 2, PP = 2, DP = 1) refitting to a single generator rank (TP = 1, PP = 1). For a representative weight tensor $\mathbf{S} = [4, 8]$ (same shape as Example A for direct comparison): PP stage 0 assigns ranks $\{0, 1\}$ in a $[1 \times 2]$ mesh with $p_s = (\text{REPLICATE}, \text{SHARD}(0))$ (TP = 2, no DP); PP stage 1 assigns ranks $\{2, 3\}$ to an identical mesh arrangement. The generator (rank 4) uses a $[1 \times 1]$ mesh with $p_d = (\text{REPLICATE})$ and must hold the full tensor for every stage's layers. Table IV lists the shard ownership for both reshard calls. After both calls complete, G_4 holds two separate full tensors of shape $[4, 8]$ —one for each PP stage's layers—whereas each trainer rank carries only the TP shard for its own stage. No five-rank communicator is required for this decomposition. Each reshard operates within a dedicated *PP communicator* scoped to its participants: PP comm 0 on global ranks $\{0, 1, 4\}$ and PP comm 1 on $\{2, 3, 4\}$, with local comm ranks 0, 1, 2 within each. The mesh's r_0 (Eq. 1) is therefore a rank within its PP communicator—rank 4 appears as comm rank 2 in both. Every rank in a given PP communicator still calls `ncclReshard` and passes both descriptors, as required by the collective contract. Because there is no destination-side replication (DP = 1 on the generator), each call reduces to a pure TP consolidation—two shards merging into one full tensor—structurally identical to a single-stage version of Example A.

Figure 2 shows the mesh decomposition. Both PP comm groups share rank 4 as their common destination; the generator mesh is constructed once and reused across both calls.

Listing 1 makes each rank's communicator membership explicit. Each trainer invokes `nccl.m2n.reshard` once, while generator rank 4 invokes it twice. Both stages share the same communicator-local layout in this example; callers may supply stage-specific layouts.

V. LAYOUT-DERIVED DESIGN AND IMPLEMENTATION

NCCL M2N separates *what* bytes move from *how* they move. From the endpoint layouts, the planner derives the

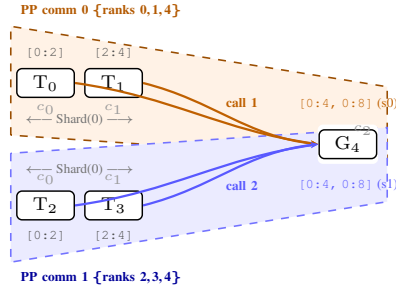


Fig. 2. PP comm structure for Example B (trainer TP=2 PP=2, generator TP=1 PP=1, global ranks 0–4). Each reshards uses its own *PP communicator*; a communicator containing all trainer stages is unnecessary. Orange funnel: PP comm 0 on global ranks {0, 1, 4} (c-ranks 0, 1, 2); blue funnel: PP comm 1 on {2, 3, 4} (same local numbering). G_4 sits at the overlap—it is c-rank 2 in both—and accumulates one full tensor per stage (s0 via call 1, s1 via call 2). The mesh’s r_0 (Eq. 1) is a rank within its PP communicator, not the global rank space.

```
import nccl.m2n as m2n

# PP communicators joined by this rank:
# ranks 0-1: [(0, pp_comm0)]
# ranks 2-3: [(1, pp_comm1)]
# rank 4: [(0, pp_comm0), (1, pp_comm1)]
for stage, pp_comm in local_stage_comms:
    src_mesh = m2n.Mesh([2], start_rank=0)
    dst_mesh = m2n.Mesh([1], start_rank=2)
    src_tensor = local_stage_weights[stage] if
    is_trainer else None
    dst_tensor = generator_weights[stage] if
    is_generator else None
    m2n.reshard(
        src_tensor, dst_tensor, pp_comm, stream=
        stream,
        src_mesh=src_mesh,
        src_placements=[m2n.Shard(0)],
        dst_mesh=dst_mesh,
        dst_placements=[m2n.Replicate()],
    )
```

Listing 1. Packaged Python API for PP decomposition. Each PP communicator uses local ranks 0–1 for its source mesh and local rank 2 for its destination mesh.

required byte ranges and topology-aware route; the implementation realizes that schedule using the mechanisms described at the end of this section.

A. Deterministic Schedule Derivation

Section IV defines each rank’s interval $[l_\delta, u_\delta]$ along tensor dimension δ . For source s and destination t , write these bounds as $[l_\delta^s, u_\delta^s]$ and $[l_\delta^t, u_\delta^t]$. Their overlap is

$$I_\delta(s, t) = [\max(l_\delta^s, l_\delta^t), \min(u_\delta^s, u_\delta^t)].$$

For each source rank s , the planner tests every destination rank t . If every $I_\delta(s, t)$ is nonempty, the combined intervals define the exact contribution; otherwise, the pair is skipped. Replicated source copies collapse to one deterministic owner. This common enumeration gives all ranks the same schedule without coordination.

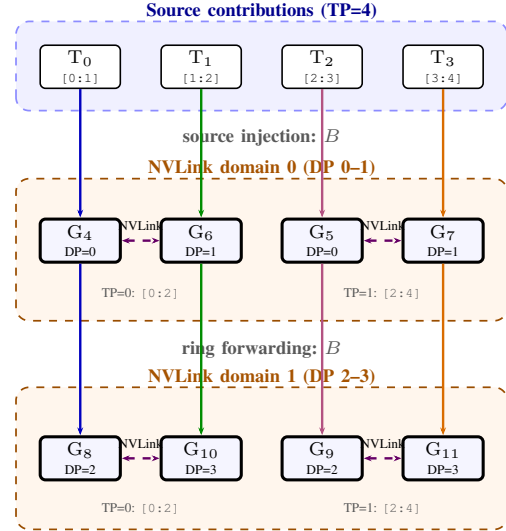


Fig. 3. Representative hierarchical route for DP=4 across two destination NVLink domains ($d = 2, \rho = 2$). Solid arrows show B -byte source injection and ring forwarding; dashed bidirectional arrows show local NVLink exchange reconstructing each TP slice on both replicas.

B. Hierarchical Topology Route

The schedule above maps each source rank to overlapping destination shards and, conversely, each destination shard to its unique source contributions. Flat direct sends transmit every contribution separately to all replicas of its destination shard. The hierarchical route instead balances contributions across selected replica leaders: each contribution enters through one leader, is ring-forwarded across domains, and is replicated locally.

Among K destination shards, let j identify one, q_j its number of unique source contributions, and ρ its local replicas per domain. Ordered by source shard, each replica receives base load $a_j = \lfloor q_j / \rho \rfloor$; the first $b_j = q_j \bmod \rho$ receive one extra. Replicas with nonzero loads become leaders and forward their contributions to the other $\rho - 1$ replicas, so loads differ by at most one.

Figure 3 extends Example A to DP=4 across two domains, where $K = 2, \rho = 2$, and $q_0 = q_1 = 2$, so $a_0 = a_1 = 1$ and $b_0 = b_1 = 0$. Thus T_0/T_1 enter through G_4/G_6 and T_2/T_3 through G_5/G_7 , balancing all four NICs rather than concentrating receives on DP0’s G_4 and G_5 . Each network stage carries B ; chunking is omitted, and traffic scales with destination domains, not replicas.

C. Aggregate Data-Movement Model

The following model gives a data-movement lower bound for the route evaluated in §VI. It intentionally omits local data preparation, chunking, protocol, and startup costs, so the bound is not a prediction of every component of end-to-end latency.

Let B be the unique global payload, N_s the number of active source GPUs, K the number of distinct destination shards, and r_d the destination replica count. The destination spans d

NVLink domains, with $\rho = r_d/d$ local replicas per shard in each domain. Let β_N and β_V be the usable one-way network and NVLink bandwidth per GPU.

Hierarchical route.: The source injects one copy of B . A destination domain has $K\rho$ active receivers, so the aggregate bandwidth of the network stage is

$$\mathcal{B}_N = \min(N_s\beta_N, K\rho\beta_N), \quad T_{\text{network}} = \frac{B}{\mathcal{B}_N}. \quad (3)$$

When the destination spans multiple domains, the inter-domain ring forwards that copy through successive domains. The network stages operate concurrently for a large payload, so d affects startup and tail latency but does not multiply the steady-state $B/\mathcal{B}_N$ term.

Within each destination domain, the network supplies one copy and NVLink creates the remaining $\rho - 1$ copies. The local copy volume is therefore $(\rho - 1)B$. Fair leader selection lets all $K\rho$ destination GPUs participate, giving

$$T_{\text{NVLink}} = \frac{(\rho - 1)B}{K\rho\beta_V}. \quad (4)$$

Network movement and local replication overlap, hence the data-movement speed-of-light (SOL) bound is simply

$$T_{\text{SOL}} = \max(T_{\text{network}}, T_{\text{NVLink}}). \quad (5)$$

Flat direct sends.: Flat direct sends inject the payload once per destination replica. Their aggregate network bound and transfer time are

$$\mathcal{B}_N^{\text{flat}} = \min(N_s\beta_N, Kr_d\beta_N), \quad T_{\text{flat}} = \frac{r_d B}{\mathcal{B}_N^{\text{flat}}}. \quad (6)$$

When source injection is the bottleneck and NVLink remains below the network term, dividing Eqs. (6) and (5) gives

$$\frac{T_{\text{flat}}}{T_{\text{SOL}}} \approx r_d. \quad (7)$$

All-gather plus broadcast.: A common framework path first all-gathers the source shards so that a designated trainer root holds the full payload, then broadcasts that payload to the destination ranks. These stages are typically pipelined across bounded buffers. Ignoring pipeline fill and drain, and assuming the two stages do not contend for resources, the ideal steady-state bound is

$$T_{\text{AG+Bcast}}^{\text{SOL}} \approx \max(T_{\text{AG}}^{\text{SOL}}, T_{\text{Bcast}}^{\text{SOL}}). \quad (8)$$

When the broadcast crosses the inter-mesh network and root injection is the bottleneck, $T_{\text{Bcast}}^{\text{SOL}} = B/\beta_N$. In the source-limited regime, M2N injects the N_s source shards concurrently, giving $T_{\text{M2N}}^{\text{SOL}} = B/(N_s\beta_N)$. Therefore,

$$T_{\text{Bcast}}^{\text{SOL}} \approx N_s T_{\text{M2N}}^{\text{SOL}}. \quad (9)$$

Here N_s is the number of independent source-injection paths; it equals the model-shard count when every shard can inject independently, and should be replaced by the effective path count when shards share NIC bandwidth.

Section VI-B instantiates these aggregate expressions and checks explicitly that NVLink is not on the critical path.

D. Implementation and Availability

The initial release (v0.2) supports one- and two-dimensional process meshes, one- to three-dimensional tensors, and at most one sharded mesh axis per layout. It realizes the derived schedule through two hierarchical transports. PACK gathers strided contributions into contiguous staging buffers, executes the hierarchical transfer, and scatters them into destination buffers. PIPE, currently beta, pipelines bounded chunks through the same route using device-side or host-driven remote-memory-access (RMA) progress. Both transports preserve caller-stream ordering. NCCL M2N is released as the `nccl_m2n` component of the open-source NVIDIA NCCL Extensions repository.¹ It provides C and packaged Python interfaces for layout-derived resharding over ordinary CUDA buffers; the repository documents the current API, build, configuration, and transport details.

VI. EVALUATION

A. Experimental Setup

Experiments use a GB200 NVL72 cluster with NDR InfiniBand (50 GB/s per NIC per direction). The single FFN-MoE layer experiment uses NCCL Extensions' `reshard_model_bench` to isolate a 7168 MB BF16 expert gate-projection transfer. A 16-GPU EP trainer group occupies one NVLink domain, while a TP8 generator group with $\text{DP} \in \{2, 4, 8\}$ (16–64 GPUs) occupies a separate domain. This expert-sharded-to-tensor-parallel transfer represents a dominant MoE weight-movement pattern. The end-to-end experiment uses DeepSeek-V3 in BF16 on 256 GPUs across 64 nodes of the same cluster. The 128 trainer GPUs span two NVL72 domains, and the 128 non-located generator GPUs span two additional domains, with 64 active GPUs per domain. Trainer parallelism is TP=1, PP=8, EP=16; generator parallelism is TP=16, DP=8. Single-layer latency is the maximum across ranks of each rank's average over 10 timed iterations following three warmups; every point also passes three correctness-validation iterations. The flat direct-send baseline sends from each active source rank to every final destination replica independently.

Measurement provenance.: The single FFN-MoE layer measurements were rerun with NCCL 2.31.0-1 and the NCCL Extensions model benchmark. They compare the flat direct-send baseline with the hierarchical route. The end-to-end experiment compares NCCL M2N against NeMo-RL's legacy all-gather-plus-broadcast path under the same non-located asynchronous RL configuration. It uses one-step off-policy execution and sequence length 1536. End-to-end metrics are reported for non-warmup execution and are labeled as step time and weight-sync time.

B. Single-Layer Microbenchmark

Figure 4 sweeps the generator replication count $r_d \in \{2, 4, 8\}$ (corresponding to 16–64 destination GPUs) while holding the active source set fixed at $m=16$ EP shards and the

¹<https://github.com/NVIDIA/nccl-extensions>

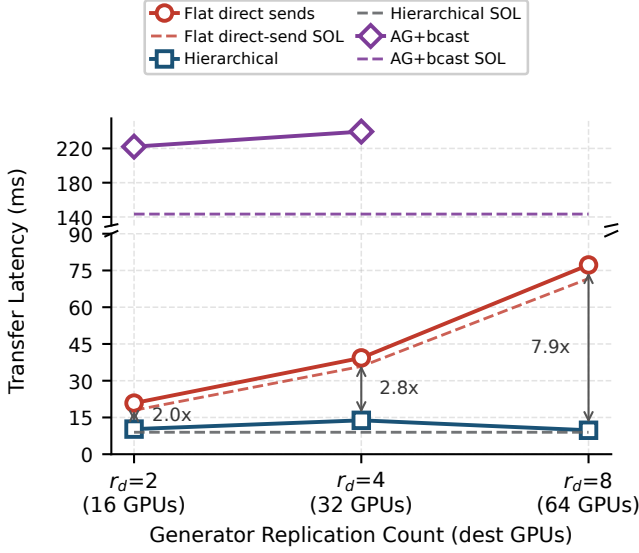


Fig. 4. Single FFN-MoE layer transfer latency on GB200 NVL72 + NDR. Trainer: TP=1, EP=16, DP={1, 2, 4}, with one node-aware active EP replica; generator: TP=8, DP= r_d (16–64 GPUs in a separate NVLink domain). Tensor: [256, 2048, 7168] BF16, 7168MB total. Dashed lines show the analytical SOL model (Eqs. (5), (6), and (8)); markers and solid lines show measured results. The broken y-axis separates the AG+bcast values from the lower-latency paths. AG+bcast measurements are included for $r_d=2$ and $r_d=4$; the $r_d=8$ point is SOL-only. Arrows annotate hierarchical speedup over flat direct sends. The Hierarchical series reports the lowest validated hierarchical-transport result at each point.

tensor parallelism at $K=8$. Additional trainer replicas make the two sides equal in allocation size but remain inactive under node-aware assignment.

Model bandwidth validation. We instantiate the aggregate equations with $B=7168$ MB, $N_s=16$, $K=8$, and $\beta_N=50$ GB/s. All three points fit in one destination NVLink domain, so the corresponding local replica counts are $\rho=\{2, 4, 8\}$. In every case,

$$\mathcal{B}_N = \min(16 \times 50, 8\rho \times 50) = 800 \text{ GB/s},$$

$$T_{\text{network}} = \frac{7168}{800} = 8.96 \text{ ms}.$$

A GB200 GPU provides 1.8TB/s bidirectional NVLink bandwidth, for which we use the one-way value $\beta_V=900$ GB/s [13]. Equation (4) then gives

$$T_{\text{NVLink}} = \frac{(\rho - 1)7168}{8\rho \times 900} = \{0.50, 0.75, 0.87\} \text{ ms}.$$

Thus NVLink is below the network term even at eight-way local replication, and Eq. (5) gives $T_{\text{SOL}} = 8.96$ ms for all three points. No per-chunk timing is needed for this bound.

Flat direct sends instead inject $r_d B$ through the same source aggregate bandwidth:

$$T_{\text{flat}} = \frac{r_d \times 7168}{800} = 8.96 r_d \text{ ms},$$

$$T_{\text{flat}} = \{17.9, 35.8, 71.7\} \text{ ms}.$$

The predicted ratio $T_{\text{flat}}/T_{\text{SOL}} = r_d$ yields the dashed curves in Figure 4.

The rank-0 AG+bcast path first all-gathers the 16 EP shards within the trainer NVLink domain, then broadcasts the full payload from one trainer root to all generator ranks. Applying Eq. (8), the ideal 16-way ring all-gather and root-injection bounds are

$$T_{\text{AG}}^{\text{SOL}} \approx \frac{15}{16} \frac{7168}{900} = 7.47 \text{ ms},$$

$$T_{\text{Bcast}}^{\text{SOL}} = \frac{7168}{50} = 143.36 \text{ ms}.$$

Assuming full pipeline overlap and no resource contention, broadcast dominates, so $T_{\text{AG+Bcast}}^{\text{SOL}} = 143.36$ ms for all three replication counts. This is $N_s=16$ times the 8.96ms M2N SOL, matching Eq. (9).

Hierarchical transport latency is nearly constant. The best validated hierarchical path completes the three transfers in {10.251, 13.834, 9.775}ms despite the $4\times$ increase in generator replication. These values are respectively 14%, 54%, and 9% above the 8.96ms SOL. The residual includes contiguous staging, pack/unpack, signalling, and startup costs. The nearly flat series confirms that adding destination replicas does not proportionally load the source NIC.

Flat direct-send latency grows with r_d . Flat direct-send latency rises from 20.859ms at $r_d = 2$ to 77.258ms at $r_d = 8$ — a $3.7\times$ increase as replication grows $4\times$. The three values are only 16%, 10%, and 8% above the corresponding {17.92, 35.84, 71.68}ms flat direct-send SOL values. This decreasing relative excess is consistent with a roughly fixed startup cost becoming a smaller fraction of the growing flat direct-send transfer volume.

Rank-0 AG+bcast is root-injection limited. For $r_d = 2$ and 4, the validated rank-0 packed path completes in 222.065 and 239.649ms, or $1.55\times$ and $1.67\times$ the 143.36ms SOL. The corresponding effective root-injection bandwidths are 32.3 and 29.9GB/s (64.6% and 59.8% of the assumed 50GB/s bandwidth). The implementation uses two bounded world broadcasts totaling 7.00GiB; splitting the payload does not change the steady-state byte bound. The $r_d = 8$ measurement is pending, so Figure 4 shows only its analytical AG+bcast SOL value.

Speedup tracks the aggregate SOL bound. Hierarchical speedup over flat direct sends grows from $2.0\times$ at $r_d = 2$ to $7.9\times$ at $r_d = 8$, with $2.8\times$ at $r_d = 4$. The endpoint values closely approach the analytical $\{2, 8\}\times$ prediction of Eq. (7); the middle point retains more transport overhead.

C. End-to-End DeepSeek-V3 RL Weight Sync

Figure 5 compares the reported weight-sync time for DeepSeek-V3 in BF16 using asynchronous NeMo-RL. Both runs use the topology described in §VI-A, sequence length 1536, one-step off-policy execution, and non-located trainer and generator groups. The comparison is between NCCL M2N and legacy all-gather plus broadcast.

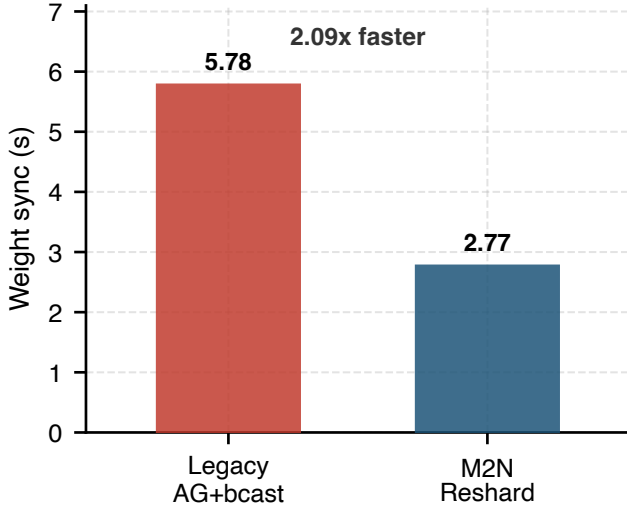


Fig. 5. DeepSeek-V3 NeMo-RL weight-sync time on 256 GB200 GPUs. Trainer: TP=1, PP=8, EP=16 (128 GPUs); generator: TP=16, DP=8 (128 GPUs). Each group spans two separate NVL72 domains, with 64 active GPUs per domain. Measurements use BF16, sequence length 1536, and one-step off-policy asynchronous execution.

Weight-sync latency.: M2N reduces the reported weight-sync time from 5.78 s to 2.77 s, a 52.1% reduction or $2.09\times$ speedup. In the baseline, weight sync accounts for 29.4% of the reported 19.68 s step time, providing the motivation in Table I.

Step time and stability.: The reported step time decreases from 19.68 s to 17.18 s (12.7%). Weight-sync time falls by 3.01 s, while step time falls by 2.50 s; the two metrics need not change by the same amount in asynchronous execution. The reported KL error is 0.001869 with M2N and 0.001912 with the legacy path. These close values provide a limited stability check for this comparison, rather than evidence of convergence or training quality.

VII. RELATED WORK

Distributed-tensor abstractions and RL training systems.: PyTorch DTensor [4], GSPMD [5], and Alpa [6] make layout a first-class abstraction but provide no cross-mesh collective. torchstore [14] and PyTorch Monarch [15] both bridge meshes through DTensor-aware transfer interfaces. torchstore additionally provides topology-based transport selection through a put/get interface. NCCL M2N addresses a different layer by deriving one global schedule from both endpoint layouts and coordinating all shard groups jointly. TensorHub [16] layers a storage abstraction (Reference-Oriented Storage) above pair-wise transport, tracking where weights live and serving topology-optimized reads on demand; NCCL M2N shares this goal of topology-awareness but operates at a different layer — a single collective call with a global, layout-derived schedule rather than a directory of per-replica fetches. NeMo-RL [7], OpenRLHF [8], HybridFlow/veRL [9], and LlamaRL [17] each implement their own refit phase using

a mix of NCCL broadcast, CUDA IPC, and one-sided DMA — efficient for their target setup, but treating refit as opaque pair-wise sends rather than as a planned topology-aware collective. NCCL M2N provides the missing primitive in this stack: an NCCL-native collective for cross-layout sharded-tensor transfer that derives a single global schedule from two (mesh, placement) endpoints at call time and plans jointly across the interconnect hierarchy — a drop-in replacement for refit and other cross-layout transfers that preserves each framework’s process-group structure.

Programmable communication abstractions.: MSCCL++ [18] exposes channel and signal abstractions over NVLink and RDMA; NVSHMEM [19] provides OpenSHMEM-style one-sided primitives. The GPU-Initiated Networking interface for NCCL [12] unifies NVLink and RDMA paths in NCCL’s device API and is the foundation our data plane builds on. NCCLX [20] scales standard collectives to 100K+ GPUs without introducing layout-aware primitives. NCCL M2N contributes the layer above these abstractions: a layout-derived schedule that tells GIN/LSA/multimem *what* to move and *which path* to use.

Offline checkpoint resharding.: Universal Checkpointing [21] saves a parallelism-agnostic checkpoint and reconstructs any target layout on load; ByteCheckpoint [22] disaggregates data and metadata for load-time resharding with $\sim 3.6\times$ speedup over manual scripts. Both target the file-system path across runs, while NCCL M2N targets in-memory transfer between two live process groups within a single run — the two approaches are complementary.

VIII. DISCUSSION AND FUTURE WORK

NCCL M2N establishes a deployable API and several transport choices, but the following directions remain open.

Multi-dimensional and higher-rank sharding.: The current implementation permits at most one sharded mesh axis per layout. Extending schedule derivation to support simultaneous sharding across multiple dimensions — and meshes with more than two axes — would cover arbitrary GSPMD-style placements and three-way parallelism combinations (e.g., TP $\times$ EP $\times$ DP) natively, without composing multiple two-dimensional meshes.

Elastic communicator APIs.: Production RL systems increasingly resize their rollout clusters dynamically. Emerging NCCL extensions for *shrink* and *grow* communicators allow process group membership to change without a full teardown and reinitialize cycle. Integrating these APIs into NCCL M2N would allow the reshard schedule to be updated in-place as the source or destination mesh changes size, making NCCL M2N a natural weight-transfer primitive for elastic RL training loops.

Adaptive transport selection.: Contiguous staging and device- or host-driven remote-memory access expose different storage and startup costs. A policy can choose among them per transfer using contiguity, tensor size, peer count, staging pressure, and whether several compatible calls can be grouped.

Such a policy should retain the current explicit environment override for reproducible evaluation.

Overlapping meshes and recovery.: Multi-rank source and destination meshes are currently disjoint, and protocol errors after host-RMA entry require communicator-level shutdown. Supporting overlapping membership would broaden in-place layout transitions; distributed failure agreement would allow safe recovery instead of the current fail-stop contract. Both require protocol changes beyond descriptor validation.

On-the-fly quantization.: Quantization is commonly applied to model weights before transfer to reduce bandwidth consumption. Rather than quantizing weights as a separate pass before resharding, NCCL M2N’s staging kernel can be extended to apply quantization transformations in-flight as chunks move through the staging ring — fusing the two operations into a single kernel pass and eliminating the intermediate quantized buffer.

IX. CONCLUSION

We presented NCCL M2N, a collective primitive for transferring distributed tensors between different source and destination layouts. By deriving the transfer schedule from both layouts, it sends only the tensor regions required by each destination and balances network traffic across destination replicas. Its hierarchical route avoids the repeated network transfers of flat direct sends and the full-tensor gathering of gather-then-broadcast, using NVLink to complete replication within each destination domain.

On a GB200 NVL72 cluster, NCCL M2N achieved up to $7.9\times$ speedup over flat direct sends for a single FFN-MoE layer transfer. In a separate 256-GPU DeepSeek-V3 NeMo-RL experiment, it reduced weight-sync time from 5.78 s to 2.77 s—a $2.09\times$ speedup over legacy all-gather plus broadcast—and reduced step time by 12.7%. These results demonstrate the benefit of planning tensor resharding jointly across layouts and the interconnect hierarchy.

REFERENCES

- [1] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training multi-billion parameter language models using model parallelism,” *arXiv:1909.08053*, 2019.
- [2] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proc. SC*, 2021.
- [3] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, 2022.
- [4] PyTorch Team, “PyTorch DTensor,” <https://pytorch.org/docs/stable/distributed.tensor.html>, 2024.
- [5] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, “GSPMD: General and scalable parallelization for ML computation graphs,” *arXiv:2105.04663*, 2021.
- [6] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, “Alpa: Automating inter- and intra-operator parallelism for distributed deep learning,” in *Proc. OSDI*, 2022.
- [7] NVIDIA, “NeMo-RL: Scalable reinforcement learning framework,” <https://github.com/NVIDIA-NeMo/RL>, 2024.

- [8] J. Hu, X. Wu, W. Shen, J. K. Liu, Z. Zhu, W. Wang, S. Jiang, H. Wang, H. Chen, B. Chen, W. Fang, Xianyu, Y. Cao, H. Xu, and Y. Liu, “OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework,” *arXiv:2405.11143*, 2024.
- [9] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu, “HybridFlow: A flexible and efficient RLHF framework,” in *Proc. EuroSys*, 2025.
- [10] “M-to-n cast primitive for PyTorch distributed (RFC),” GitHub Issue [pytorch/pytorch#181061](https://github.com/pytorch/pytorch/issues/181061), 2026.
- [11] NVIDIA, “NVIDIA collective communication library (NCCL),” <https://github.com/NVIDIA/nccl>.
- [12] K. Hamidouche, J. Bachan, P. Markthub, P.-J. Gootzen, E. Agostini, S. Jeaugey, A. Shafi, G. Theodorakis, and M. G. Venkata, “GPU-initiated networking for NCCL,” *arXiv:2511.15076*, 2025.
- [13] NVIDIA, “Networking,” NVIDIA DGX GB Rack Scale Systems User Guide, <https://docs.nvidia.com/dgx/dgxb200-user-guide/networking.html>, 2026.
- [14] Meta PyTorch, “torchstore: Weight storage for distributed PyTorch,” <https://github.com/meta-pytorch/torchstore>, 2024.
- [15] —, “Introducing PyTorch Monarch: Single-controller distributed programming for PyTorch,” <https://pytorch.org/blog/introducing-pytorch-monarch/>; source at <https://github.com/meta-pytorch/monarch>, 2025.
- [16] C. Ye, H. Zhang, M. Han, B. Zhong, X. Li, Q. Chen, X. Zhang, W. Zhang, K. Jiang, W. Zhang, H. Sun, W. Xiao, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “TensorHub: Scalable and elastic weight transfer for LLM RL training,” *arXiv:2604.09107*, 2026.
- [17] B. Wu, S. Wang, Y. Tang, J. Ding, E. Helenowski, L. Tan, T. Xu, T. Gowda, Z. Chen, C. Zhu, X. Tang, Y. Qian, B. Zhu, and R. Hou, “LlamaRL: A distributed asynchronous reinforcement learning framework for efficient large-scale LLM training,” *arXiv:2505.24034*, 2025.
- [18] C. Hwang, P. Cheng, R. Dathathri, A. Jangda, S. Maleki, M. Musuvathi, O. Saarikivi, A. Shah, Z. Yang, B. Li, C. Rocha, Q. Zhou, M. Ghazimirsaeed, S. Anantharamu, and J. Jose, “MSCCL++: Rethinking GPU communication abstractions for AI inference,” in *Proc. ASPLOS*, 2026.
- [19] NVIDIA, “NVSHMEM: GPU-initiated communication based on the OpenSHMEM standard,” <https://developer.nvidia.com/nvshmem>.
- [20] M. Si, P. Balaji, Y. Chen, C.-H. Chu, A. Gangidi, S. Hasan, S. Iyengar, D. Johnson, B. Liu, R. Ren, A. J. Shetty, G. Steinbrecher, Y. Wang, B. Wu, X. Xie, J. Yang, M. Yang, K. Yu, M. Yu, C. Zhao, W. Bland, D. Boyda, S. Gumudavelli, P. Kannan, C. Lumezanu, R. Miao, Z. Qu, V. Ramesh, M. Samoylov, J. Seidel, S. Sundaresan, F. Tian, Q. Tan, S. Zhang, Y. Zhao, S. Zheng, A. Zhu, and H. Zeng, “Collective communication for 100k+ GPUs,” *arXiv:2510.20171*, 2025.
- [21] X. Lian, S. A. Jacobs, L. Kurilenko, M. Tanaka, S. Bekman, O. Ruwase, and M. Zhang, “Universal Checkpointing: A flexible and efficient distributed checkpointing system for large-scale DNN training with reconfigurable parallelism,” in *Proc. USENIX ATC*, 2025.
- [22] B. Wan, M. Han, Y. Sheng, Y. Peng, H. Lin, M. Zhang, Z. Lai, M. Yu, J. Zhang, Z. Song, X. Liu, and C. Wu, “ByteCheckpoint: A unified checkpointing system for large foundation model development,” in *Proc. NSDI*, 2025.